

Neuromorphic ASR

Transducer Models on Memristor Hardware

Azim Javed

azim.javed@ml.rwth-aachen.de

September 7, 2026

Advisor: Benedikt Hilmes and Simon Berger

Supervisor: Prof. Ralf Schlüter

Human Language Technology and Pattern Recognition
Computer Science Department, RWTH Aachen University

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

In-Memory Computing [Rossenbach & Hilmes⁺ 25]

- ▶ Various technologies exist, with varying tradeoffs:
 - ▷ Chip complexity and density
 - ▷ Energy consumption
 - ▷ Speed and reliability
- ▶ This talk focuses on **memristive** hardware, in particular **resistive RAM (ReRAM)**:
 - ▷ Key benefits: low energy consumption, non-volatility

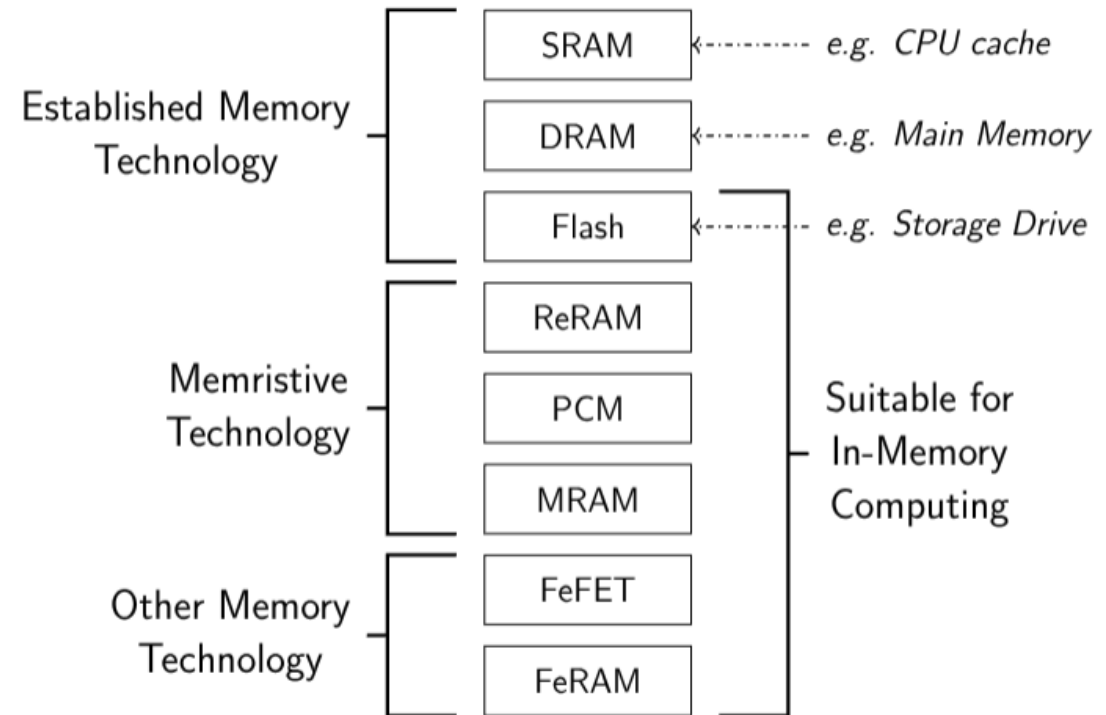


Figure: Taxonomy of In-Memory Computing technologies

Overview [Rossenbach & Hilmes⁺ 25]

- ▶ A **memristor** is an analog computing device with a programmable resistance state [Strukov & Snider⁺ 08].
- ▶ Enables fast, energy-efficient implementation of Vector-Matrix Multiplication (VMM) [Wan & Kubendran⁺ 22] —
→ and thus neural networks.
- ▶ Limitations:
 - ▷ High analog inaccuracies and thus need specific restrictions on mathematical operations [Wouters & Chen⁺ 16].
 - ▷ Production is still in early stages [Infineon Technologies AG 22].
 - ▷ Current experiments are simulated using SynaptogenML [Hennen & Brackmann⁺ 24] [Rossenbach & Hilmes⁺ 25].

Resistor:



Memristor:



Figure: Resistor and memristor schematics

Analog In-Memory Computing [Aguirre & Sebastian⁺ 24]

- Multiplication computation done via Ohm's law:

$$V = IR \quad (1)$$

- Voltage V , current I , and resistance R .
- Possible to rewrite resistance as conductance G :

$$I = GV \quad \text{with} \quad G = \frac{1}{R} \quad (2)$$

- Addition done via Kirchhoff's law for any connected node in a circuit:

$$\sum_{n=0}^N I_n = 0 \rightarrow I_0 = - \sum_{n=1}^N I_n \quad (3)$$

- Currents can be trivially added by simply connecting electric lines.

Analog In-Memory Computing [Rossenbach & Hilmes⁺ 25]

- ▶ Resistors ($\square$) as connecting element between vertical and horizontal circuit lines
- ▶ Each resistor's conductance is an element of a constant matrix
- ▶ Column sum simply via line connection

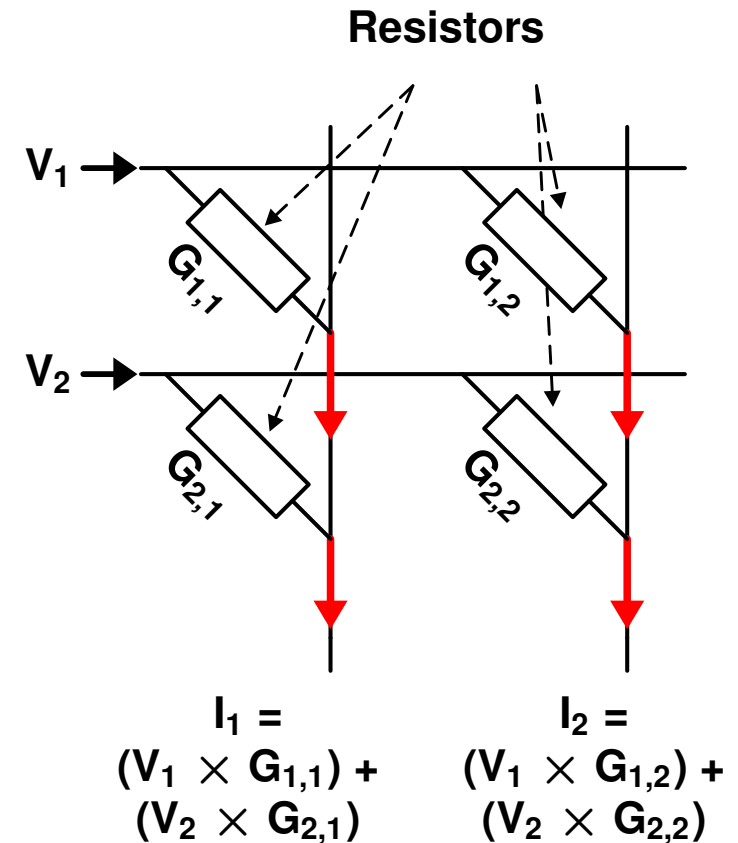


Figure: Crossbar array performing analog VMM [Rossenbach & Hilmes⁺ 25]

The Memristor [Hennen & Brackmann⁺ 24]

Key Idea: Program weights by setting the resistance.

- ▶ Apply $-V > -V_S \rightarrow$ set to low resistance
- ▶ Apply $V > V_R \rightarrow$ set to high resistance
- ▶ Intermediate states possible, but difficult to program reliably
- ▶ State is physically stored (non-volatile memory)

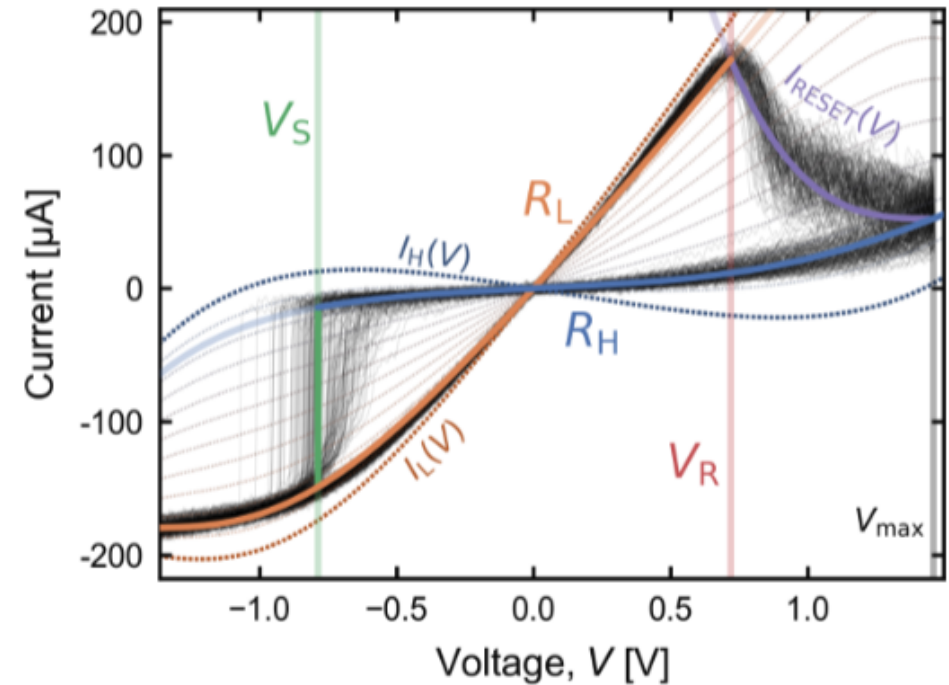


Figure: I–V characteristic curve of a memristor [Hennen & Brackmann⁺ 24]

Limitations [Rossenbach & Hilmes⁺ 25]

Programming Limitations

- ▶ Resulting resistance state varies with each programming cycle
- ▶ Non-linear relation of current and voltage

Cannot exactly model weights

- ▶ Only positive, non-zero weights possible
- ▶ Current leakage prohibits a true zero

(Paired) Memristor Array [Aguirre & Sebastian⁺ 24]

- ▶ Use positive and negative bitlines to create memristor pairs
- ▶ Each row-wise pair of memristors represents a weight
- Differential pair allows for negative and zero weight values:

$$I_j = \sum_{i=1}^N V_i G_{i,j}^+ - \sum_{i=1}^N V_i G_{i,j}^-$$

Problems:

- ▶ Cannot set paired memristors to identical conductances
- Difference close to zero, but not guaranteed

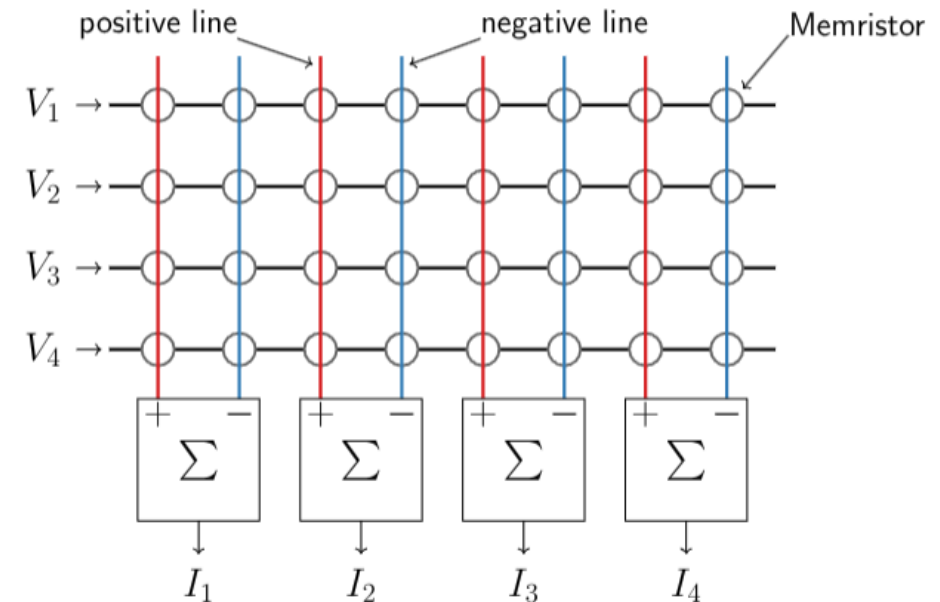


Figure: Paired memristor array structure [Aguirre & Sebastian⁺ 24]

Memristor Neural Layers [Rossenbach & Hilmes⁺ 25]

- ▶ **Tile** the weight matrix:
 - ▷ Tile weight matrix with arbitrary dimensions into smaller fixed-size arrays.
 - ▷ E.g., 512×512 weight matrix $\rightarrow$ 16 crossbar arrays of size 128×128 .
- ▶ **Slice** weights into memristor states:
 - ▷ Binary setting (high vs. low resistance).
 - ▷ Stacking of memristors for each magnitude bit (k -bit weight requires $k - 1$ arrays).
- ▶ **Pair** positive (+) and negative (−) bitlines.

Memristor Neural Layers

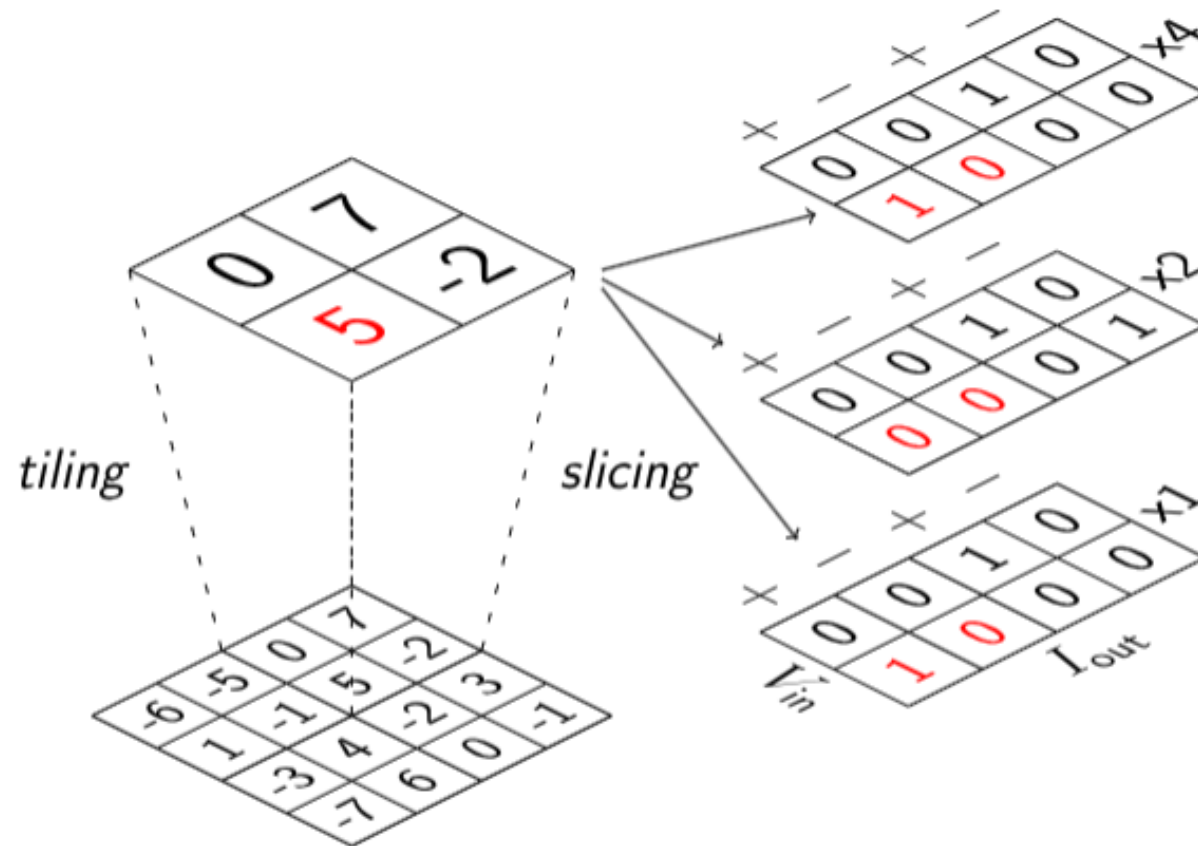


Figure: Mapping a weight matrix on memristor arrays [Rossenbach & Hilmes⁺ 25]

Quantization

- ▶ The number of memristors required is proportional to the precision of the weights.
 - ▷ Need to reduce the precision of the weights → quantization.
- ▶ Quantization-Aware Training (QAT) performs better than Post-Training Quantization (PTQ), but requires more training time [Gholami & Kim⁺ 21].
 - High performance gap at low precision

Table: WER (%) on TED-LIUMv2 dev, Conformer CTC, 4-gram LM [Rossenbach & Hilmes⁺ 25]

Weight bits	FP32	8	6	5	4	3	2
PTQ	7.2	7.2	7.9	8.9	11.4	30.7	–
QAT		7.3	7.7	7.4	7.8	8.3	22.1

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

Motivation

- ▶ CTC-based Conformer shown to run on (simulated) memristor hardware [Rossenbach & Hilmes⁺ 25].
- ▶ How do **transducer** models, with their added complexity compared to CTC, behave on device?
 - ▷ Autoregressive prediction network; internal language model (ILM).
 - ▷ Joint network evaluated repeatedly per emitted label.
- ▶ Performance degradation across search paradigms:
 - ▷ Lexicon-free vs. tree search.
 - ▷ ILM and prior scaling — How does device noise change the optimal LM weighting?

Motivation

- ▶ Performance impact of deploying components of the model on the device?
 - ▷ Encoder only vs. encoder + prediction network vs. full model (encoder + prediction network + joint network)
- ▶ **Tradeoff**: more offloading $\Rightarrow$ more energy efficiency, but also more noise.

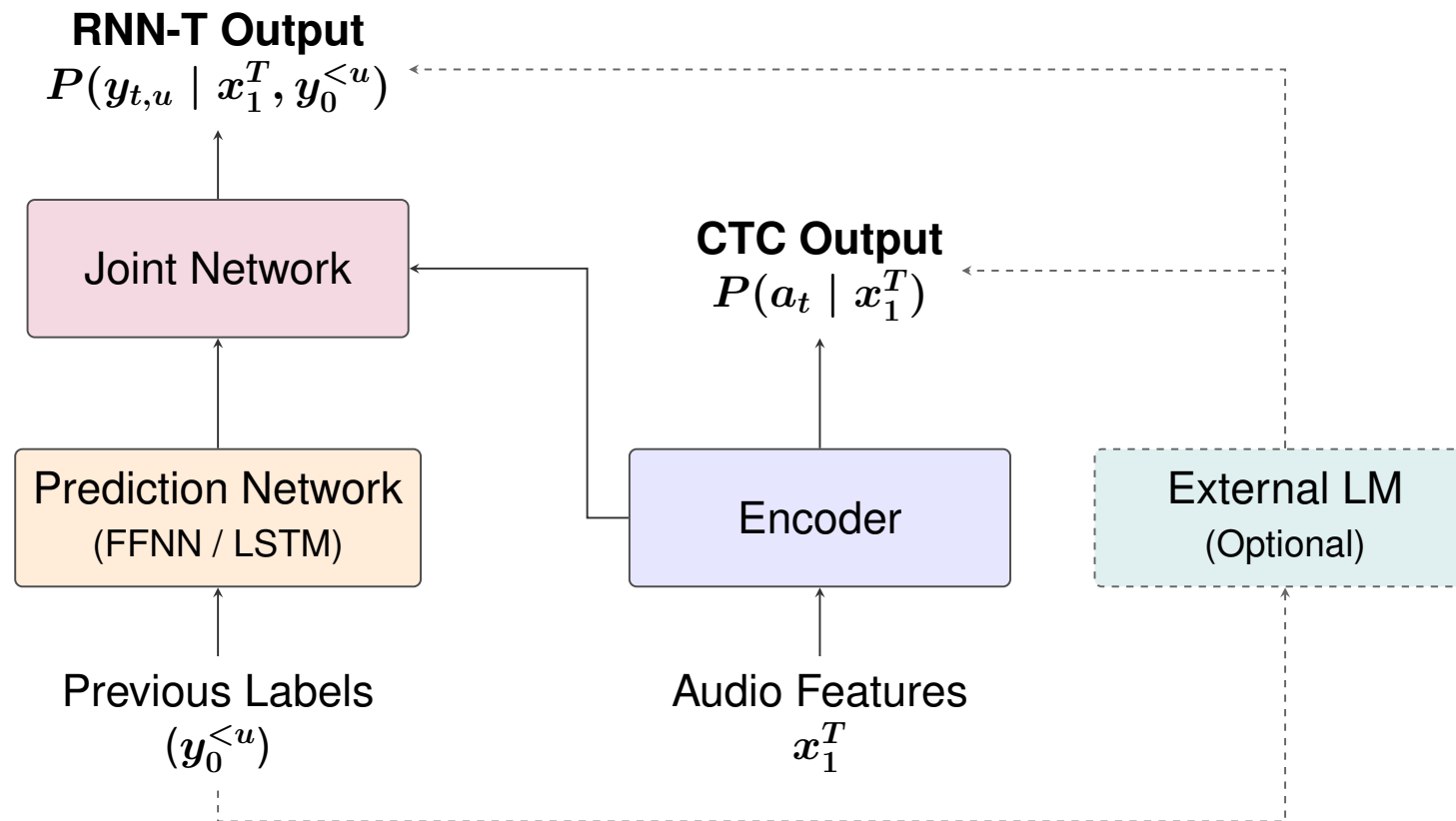


Figure: CTC vs. Transducer architecture components

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

Experimental Setup

- ▶ Data: LibriSpeech [Panayotov & Chen⁺ 15] (960 h), BPE subword targets (128 merges → 184 labels).
- ▶ Features: 80 log-mel filters, 10 ms frame shift; SpecAugment during training.
- ▶ Encoder: 12-layer Conformer [Gulati & Qin⁺ 20], 512-dim.
- ▶ Model types:
 1. **CTC** [Graves & Fernández⁺ 06]: encoder only.
 2. **FFNN-transducer** [Graves 12]: prediction network is a 2-layer feed-forward net (640-dim) over the previous-label embedding; monotonic alignments [Tripathi & Lu⁺ 19].
 3. **Full-context transducer** [Graves 12]: LSTM prediction network; monotonic alignments [Tripathi & Lu⁺ 19].
- ▶ Quantization: QAT, per-tensor symmetric, 8- and 4-bit weights, 8-bit activations; zero-point 0.
- ▶ Stages: (1) Full-precision baseline → (2) Full pre-training with QAT (fake quantization) → (3) memristor simulation (3–5 runs).

CTC Experiments

Table: WER (%) on LibriSpeech dev-other – CTC

(a) Baseline

Model	Lex-Free	Tree	
	No LM	No LM	4-gram
CTC	7.1	5.0	5.2

(b) QAT

Bits	Lex-Free	Tree	
	No LM	No LM	4-gram
8	7.3	5.2	5.3
4	8.3	5.4	5.7

(c) Memristor Simulation

Bits	Lex-Free	Tree	
	No LM	No LM	4-gram
8	13.5 ± 0.37	10.4 ± 0.20	6.7 ± 0.09
4	11.9 ± 0.18	TBD	6.8 ± 0.12

FFNN Transducer Experiments: 8-bit Quantization

Table: WER (%) on LibriSpeech dev-other (8-bit Quantization vs. baseline)

Modules Quantized			Lex-Free	Tree	
Encoder	Predictor	Joiner	No LM	No LM	4-gram
Baseline					
—	—	—	5.9	5.8	4.8
QAT					
✓	—	—	6.3	6.1	4.9
✓	✓	—	6.4	6.2	5.0
✓	✓	✓	6.4	6.2	4.9
Memristor Simulation					
✓	—	—	9.8 ± 0.06	8.7 ± 0.03	6.7 ± 0.07
✓	✓	—	10.7 ± 0.10	8.9 ± 0.11	6.7 ± 0.04
✓	✓	✓	TBD	TBD	TBD

FFNN Transducer Experiments: 4-bit Quantization

Table: WER (%) on LibriSpeech dev-other (4-bit Quantization vs. baseline)

Modules Quantized			Lex-Free	Tree	
Encoder	Predictor	Joiner	No LM	No LM	4-gram
Baseline					
—	—	—	5.9	5.8	4.8
QAT					
✓	—	—	7.2	6.8	5.2
✓	✓	—	TBD	TBD	TBD
✓	✓	✓	TBD	TBD	TBD
Memristor Simulation					
✓	—	—	10.1 ± 0.05	9.1 ± 0.11	6.7 ± 0.07
✓	✓	—	TBD	TBD	TBD
✓	✓	✓	TBD	TBD	TBD

FFNN Transducer: Fine-Tuning vs. QAT from Scratch

Table: WER (%) on LibriSpeech dev-other: Checkpoint fine-tuning vs. full pre-training (PTN).
Configuration – FFNN Transducer with Encoder under QAT

(a) QAT				
Training	Bits	Lex-Free	Tree	
		No LM	No LM	4-gram
Fine-tuned	8	6.6	6.3	5.0
Full PTN	8	6.5	6.2	5.1
Fine-tuned	4	7.2	7.0	5.1
Full PTN	4	7.2	6.8	5.2

(b) Memristor Simulation				
Training	Bits	Lex-Free	Tree	
		No LM	No LM	4-gram
Fine-tuned	8	8.1 ± 0.06	7.5 ± 0.07	5.8 ± 0.08
Full PTN	8	9.8 ± 0.06	8.7 ± 0.03	6.7 ± 0.07
Fine-tuned**	4	13.1 ± 0.06	11.2 ± 0.10	7.2 ± 0.05
Full PTN	4	10.1 ± 0.05	9.1 ± 0.11	6.7 ± 0.07

**Tentative results, to be updated.

→ Fine-tuning from a full-precision checkpoint is more effective than training from scratch for memristor simulation.

Full-Context Transducer Experiments: 8-bit Quantization

Table: WER (%) on LibriSpeech dev-other (8-bit Quantization vs. baseline)

Modules Quantized			Lex-Free	Tree	
Encoder	Predictor	Joiner	No LM	No LM	4-gram
Baseline					
—	—	—	5.8	5.6	4.9
QAT					
✓	—	—	5.9	5.7	5.2
✓	✓	—	6.1	5.9	5.2
✓	✓	✓	6.1	5.9	5.2
Memristor Simulation					
✓	—	—	9.0 ± 0.29	8.1 ± 0.16	7.1 ± 0.07
✓	✓	—	WIP	WIP	WIP
✓	✓	✓	WIP	WIP	WIP

Noisier Models Demand Stronger Internal LM Compensation

Table: WER (%) on LibriSpeech (dev-other, ELM scale = 0.6, Tree + 4-gram LM search).

	Modules Quantized		ILM Scale			
	Encoder	Predictor	0.1	0.2	0.3	0.4
Baseline	–	–	4.8	4.8	4.8	5.0
QAT	✓	–	5.1	5.1	5.1	5.3
	✓	✓	5.0	5.0	5.1	5.2
Memristor	✓	–	17.4 ± 1.60	8.5 ± 0.34	7.0 ± 0.06	6.8 ± 0.08
	✓	✓	18.7 ± 1.58	8.8 ± 0.23	7.2 ± 0.09	6.8 ± 0.03

Table: Optimal LM scales and WER (%) on LibriSpeech (dev-other, Tree + 4-gram LM search).

	Modules Quantized		Scales		
	Encoder	Predictor	ILM	ELM	WER (%)
Baseline	–	–	0.1	0.6	4.8
QAT	✓	–	0.2	0.7	5.0
	✓	✓	0.2	0.7	5.0
Memristor	✓	–	0.3	0.5	6.7 ± 0.07
	✓	✓	0.4	0.5	6.7 ± 0.04

ILM: Internal Language Model; **ELM:** External Language Model.

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

Future Work: Error Analysis

► Fine-tuning:

- ▷ Fine-tuning from a full-precision checkpoint for other configurations to study the impact on WER.

► Error Analysis:

- ▷ Substitutions vs. insertions vs. deletions.
 - Why does lexicon-free search degrade more than tree search?
- ▷ Acoustic posterior distortion and score calibration with external LMs.

► Generalization:

- ▷ Evaluation and training on larger/out-of-domain datasets.

Future Work: Search

► Decoder Parameters:

- ▷ Explore penalties for blank emissions.
- ▷ Tune softmax temperature to sharpen or soften posterior peaks under noise.

► Search Space vs. Model & QAT Level:

- ▷ Large search space $\Rightarrow$ more hypotheses $\Rightarrow$ more energy consumption and higher RTF.
- ▷ Study the impact of restricting the search space on WER and RTF.
 - $\rightarrow$ How many more hypotheses are needed for on-device vs. 4-bit vs. 8-bit vs. full-precision runs?
 - $\rightarrow$ Compare hypothesis expansion across CTC and transducer variants.

► Search Paradigms:

- ▷ Explore alternative search algorithms and language model combinations (e.g., lexicon-free with LSTM/transformer-based LM).

Future Work: Robust Training & Countermeasures

► Noise-Aware QAT:

- ▷ Inject simulated crossbar conductance noise directly into training forward pass.

► Selective Layer Mapping:

- ▷ Deal with noise in noise-sensitive components (e.g., joint network).
→ Mixed-precision quantization

► Denoising & Output Compensation:

- ▷ Post-processing to correct analog inaccuracies.
→ Train a small neural network to denoise the memristor output.
→ Use a small calibration dataset to learn a linear transformation to correct the output.

Thank you for your attention!

References

-  F. Aguirre, A. Sebastian, M. Le Gallo, W. Song, T. Wang, J. J. Yang, W. D. Lu, M.-F. Chang, D. Ielmini, Y. Yang, A. Mehonic, A. J. Kenyon, M. A. Villena, J. B. Roldán, Y. Wu, H.-H. Hsu, N. Raghavan, J. Suñé, E. Miranda, A. Eltawil, G. Setti, K. Smagulova, K. N. Salama, O. Krestinskaya, X. Yan, K.-W. Ang, S. Jain, S. Li, O. Alharbi, S. Pazos, M. Lanza.
Hardware Implementation of Memristor-Based Artificial Neural Networks.
Nature Communications, Vol. 15, No. 1, pp. 1974, March 2024.
-  A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, K. Keutzer.
A Survey of Quantization Methods for Efficient Neural Network Inference, Aug. 2021.
[arXiv:2103.13630](https://arxiv.org/abs/2103.13630).
-  A. Graves, S. Fernández, F. Gomez, J. Schmidhuber.
Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks.
In *Proc. ICML*, pp. 369–376, Pittsburgh, PA, June 2006.

- 📄 A. Graves.
Sequence Transduction with Recurrent Neural Networks.
In *Proc. ICML*, Edinburgh, Scotland, June 2012.
Workshop on Representation Learning, arXiv:1211.3711.
- 📄 A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, R. Pang.
Conformer: Convolution-Augmented Transformer for Speech Recognition.
In *Proc. Interspeech*, pp. 5036–5040, Shanghai, China, Oct. 2020.
- 📄 T. Hennen, L. Brackmann, T. Ziegler, S. Siegel, S. Menzel, R. Waser, D. J. Wouters, D. Bedau.
Synaptogen: A Cross-Domain Generative Device Model for Large-Scale Neuromorphic Circuit Design.
IEEE Transactions on Electron Devices, Vol. 71, No. 9, pp. 5345–5353, Sept. 2024.
arXiv:2404.06344.

- 📄 **Infineon Technologies AG.**
Infineon and TSMC to Introduce RRAM Technology for Automotive AURIX™ TC4x Product Family.
<https://www.infineon.com/cms/de/about-infineon/press/market-news/2022/INFATV202211-031.html>, Nov. 2022.
- 📄 **V. Panayotov, G. Chen, D. Povey, S. Khudanpur.**
Librispeech: an ASR Corpus Based on Public Domain Audio Books.
In *Proc. IEEE ICASSP*, pp. 5206–5210, Queensland, Australia, April 2015.
- 📄 **N. Rossenbach, B. Hilmes, L. Brackmann, M. Gunz, R. Schlüter.**
Running Conventional Automatic Speech Recognition on Memristor Hardware: A Simulated Approach.
In *Proc. Interspeech*, Rotterdam, The Netherlands, Aug. 2025.
[arXiv:2505.24721](https://arxiv.org/abs/2505.24721).
- 📄 **D. B. Strukov, G. Snider, D. R. Stewart, R. S. Williams.**
The missing memristor found.
Nature, Vol. 453, pp. 80–83, 2008.

- 📄 A. Tripathi, H. Lu, H. Sak, H. Soltau.
Monotonic Recurrent Neural Network Transducer and Decoding Strategies.
In *Proc. IEEE ASRU*, pp. 944–948, Sentosa, Singapore, Dec. 2019.
- 📄 W. Wan, R. Kubendran, C. Schaefer, S. B. Eryilmaz, W. Zhang, D. Wu, S. Deiss, P. Raina, H. Qian, B. Gao, S. Joshi, H. Wu, H.-S. P. Wong, G. Cauwenberghs.
A Compute-in-Memory Chip Based on Resistive Random-Access Memory.
Nature, Vol. 608, No. 7923, pp. 504–512, Aug. 2022.
- 📄 D. J. Wouters, Y.-Y. Chen, A. Fantini, N. Raghavan.
Reliability Aspects.
In D. Ielmini, R. Waser, editors, *Resistive Switching: From Fundamentals of Nanoionic Redox Processes to Memristive Device Applications*, chapter 21, pp. 597–622. John Wiley & Sons, Ltd, 2016.

Appendix: Device VMM Operation Statistics

operation	average	std-dev	min	max
1.0×1	1.016	0.063	0.707	1.27
0.1×1	$0.989 \cdot 10^{-1}$	$0.062 \cdot 10^{-1}$	$0.688 \cdot 10^{-1}$	$1.24 \cdot 10^{-1}$
0.01×1	$0.985 \cdot 10^{-2}$	$0.069 \cdot 10^{-2}$	$0.721 \cdot 10^{-2}$	$1.20 \cdot 10^{-2}$
1.0×0	$8.52 \cdot 10^{-4}$	$4.75 \cdot 10^{-2}$	-0.2751	0.2455
1.0×0.5	0.476	0.094	0.109	1.06

Table: Hardware measurement statistics for simulated memristor VMM operations [Rossenbach & Hilmes⁺ 25].